Arduino Nano Based PID Line Following Robot for Autonomous Navigation

Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering of the University of Asia Pacific

by

Tanjuma Noor
Roll: 22201210

Muhib Al Ahad
Roll: 22201207

Minazur Rahman Minhaz
Roll: 22201208

Supervised by:
Nazmun Nahid
Assistant Professor

Department of Computer Science & Engineering
University of Asia Pacific

June 2026

Declaration

We, hereby, declare that the work presented in this report is the outcome of the investigation performed by us under the supervision of Nazmun Nahid, Assistant Professor, Department of Computer Science and Engineering, University of Asia Pacific. We also declare that no part of this report and thereof has been or is being submitted elsewhere for the award of any degree or Diploma.

Countersigned

(Nazmun Nahid)
Supervisor

Signature

.....

Tanjuma Noor

ID: 22201210

.....

Muhib Al Ahad

ID: 22201207

.....

Minazur Rahman Minhaz

ID: 22201208

Abstract

Line following robots are a widely used entry point into autonomous mobile robotics, and the quality of their path-tracking behaviour depends heavily on the controller that converts sensor readings into motor commands. This report presents the design, implementation, and evaluation of an Arduino Nano based line following robot that uses a closed-loop Proportional-Integral-Derivative (PID) controller to follow a black line on a white surface. The robot uses a 5-channel TCRT5000 infrared (IR) sensor array to compute a weighted cross-track error, which is processed by a PID control loop running on an Arduino Nano to generate a steering correction. This correction is converted into differential PWM signals and sent to an L298N dual H-bridge motor driver, which drives two 400 RPM N20 DC geared motors. Power is supplied by two 18650 Li-ion cells regulated through a buck converter to a stable 5 V logic rail. The PID gains ($K_p = 7$, $K_i = 0.001$, $K_d = 0.4$) were empirically tuned to minimise oscillation while maintaining responsiveness at a base speed of 60 (PWM duty value), with the control loop executing at a fixed 50 ms sampling interval. The system also implements a basic fault recovery routine (pivot/search sweep) for momentary line loss. Testing on a physical track showed smooth, low-oscillation tracking with reliable recovery at corners and minor gaps in the line. The report documents the complete hardware design, the algorithmic flow and pseudocode, the firmware implementation, project planning, and budget, and concludes with the limitations of the current prototype and recommendations for future work, including an 8-sensor upgrade, closed-loop motor speed control, and improved low-light filtering.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Autonomous mobile robots are increasingly used in education, warehouse automation, and robotics competitions, and the line following robot (LFR) is one of the most common starting platforms for learning closed-loop control. An LFR uses an array of optical sensors to detect a contrasting line on the floor (typically a black line on a white surface, or vice versa) and adjusts its motor speeds so that it stays centred on the line as it moves forward.

Simple LFR designs use only on/off (bang-bang) logic, where the robot turns sharply whenever it drifts off the line. This produces jerky, oscillatory motion and poor performance at speed. A Proportional-Integral-Derivative (PID) controller instead computes a continuous correction signal that is proportional to how far off-line the robot is, how long it has been off-line, and how quickly the error is changing. This produces smoother, faster, and more stable path tracking, which is particularly important in line following competitions where lap time and stability are scored.

This project builds an Arduino Nano based line following robot that follows a black line on a white surface using a 5-channel IR sensor array and a closed-loop PID controller. The robot's control loop continuously reads the sensors, computes a weighted position error, runs the PID algorithm to generate a correction value, and applies that correction as a differential PWM signal to the left and right drive motors through an L298N motor driver.

1.2 Problem Statement

Many low-cost line following robot builds rely purely on proportional or on/off control, which is simple to implement but performs poorly at higher speeds and on tracks with sharp curves: the robot overshoots, oscillates around the line, or loses the line entirely at corners. Achieving smooth, low-oscillation tracking that remains stable at competition speeds requires a full PID controller with carefully tuned gains, a reliable sensor calibration procedure, and a fault-recovery mechanism for the cases where the line is briefly lost. There is a need for an affordable, reproducible hardware and firmware design that demonstrates this complete closed-loop control pipeline using widely available components (Arduino Nano, L298N, TCRT5000 IR array), rather than more expensive or complex platforms.

1.3 Objectives of the Project

1. To design and assemble the hardware of a line following robot using an Arduino Nano, an L298N motor driver, a 5-channel TCRT5000 IR sensor array, two 400 RPM N20 DC motors, and a 2×18650 Li-ion battery pack regulated through a buck converter.

2. To implement a closed-loop PID controller on the Arduino Nano that computes a weighted cross-track error from the IR sensor array and converts it into a steering correction.

3. To tune the PID gains (K_p, K_i, K_d) empirically to minimise oscillation and overshoot while maintaining responsive tracking at competition speed.

4. To implement a basic fault-recovery (search/pivot) behaviour for momentary loss of the line.

5. To evaluate the robot's tracking performance on a physical test track and document the complete system design, implementation, and results.

1.4 Scope of the Work

This project covers the complete hardware assembly and firmware development of a single line following robot prototype, including sensor calibration, PID control loop implementation, motor driver integration, and power supply design. The scope is limited to following a single, continuous high-contrast line; it does not include junction/intersection decision-making, maze solving, wireless telemetry, or machine-learning-based line detection, although several of these are discussed as future work. Physical chassis fabrication uses a simple 3D-printed sensor mount and a breadboard-based electronics layout rather than a custom PCB.

1.5 Expected Outcomes

- A working robot prototype that follows a black line on a white track with low lateral error and minimal oscillation at speed.
- A documented PID control firmware implementation, including the weighted error calculation, the PID computation, and the motor speed mapping logic.
- A reproducible hardware design with a documented pin configuration, bill of materials, and assembly notes.
- Demonstrated fault-recovery behaviour when the robot momentarily loses the line.

1.6 Impacts of the Project

1.6.1 Impact on Societal and Cultural Issues

Line following robots are a foundational platform used in robotics competitions and STEM education across schools and universities in Bangladesh and elsewhere. A well-documented, low-cost PID-based design such as this one lowers the barrier for students and hobbyists to learn practical closed-loop control theory through a hands-on project, rather than purely through abstract coursework.

1.6.2 Impact on Health, Safety, and Legal Issues

The robot operates from a low-voltage 7.4 V (2×18650) battery pack regulated to 5 V logic, which poses minimal electrical shock risk. The main safety consideration is the Li-ion battery pack, which can pose a fire or swelling hazard if short-circuited, physically damaged, or overcharged; this is mitigated by using batteries with built-in protection circuitry and by correctly rating the buck converter for the motor and logic loads. The robot does not collect, store, or transmit any personal data, so it raises no privacy or legal concerns.

1.6.3 Impact on Environment and Sustainability Issues

The robot is powered by rechargeable 18650 Li-ion cells rather than disposable batteries, reducing battery waste over repeated use and testing. Component selection favours widely available, low-cost, and individually replaceable parts (Arduino Nano, L298N, TCRT5000 array, N20 motors), which supports repairability and reduces the environmental footprint associated with discarding the entire device if a single component fails.

Chapter 2

Background and Literature Review

2.1 Preliminaries

Line Following Robot

A line following robot (LFR) is a mobile robot that autonomously tracks a visually contrasting path — typically a black line on a white surface, or vice versa — using an array of optical sensors. The robot continuously measures how far it has drifted from the line and adjusts the speed of its left and right wheels to steer back towards it. LFRs are widely used as an introductory platform for closed-loop control, and the same underlying control approach scales to industrial automated guided vehicles (AGVs) used for material transport.

Infrared (IR) Reflectance Sensing

IR reflectance sensors consist of an IR emitter and an IR phototransistor or photodiode receiver mounted side by side. Light surfaces reflect more IR light back to the receiver than dark surfaces, producing a measurable difference in output voltage (or analog reading) between a sensor positioned over the line and one positioned over the background. The TCRT5000 module used in this project is a common, low-cost implementation of this principle, and is typically arranged in a linear array of several such sensors to give the controller a spatial view of the line's position relative to the robot.

PID Control

A Proportional-Integral-Derivative (PID) controller is a closed-loop feedback algorithm that computes a control output as a weighted sum of three terms: the proportional term (responds to the present error), the integral term (responds to the accumulated past error, correcting persistent bias), and the derivative term (responds to the rate of change of the error, anticipating overshoot). In a line following robot, the ”error” is the lateral displacement of the robot from the centre of the line, and the PID output is used to bias the speed of one motor relative to the other so that the robot steers back towards the line.

Differential Drive and PWM Motor Control

A differential drive robot has two independently driven wheels, and steers by varying the relative speed of the left and right wheels rather than using a separate steering mechanism. Motor speed is controlled using Pulse Width Modulation (PWM), where the average voltage delivered to the motor is varied by changing the duty cycle of a fixed-frequency square wave. An H-bridge motor driver (such as the L298N used in this project) allows a low-power microcontroller to switch the high-current motor supply in both directions, enabling forward, reverse, and braking control of each motor independently.

Arduino Nano Microcontroller

The Arduino Nano is a compact, breadboard-friendly microcontroller board based on the ATmega328P. It provides analog inputs (used here to read the IR sensor array), PWM-capable digital outputs (used to drive the motor driver), and is programmed using the Arduino IDE in C/C++. Its low cost, small footprint, and large hobbyist ecosystem make it a popular choice for small mobile robotics projects such as this one.

2.2 Related Works / Existing Systems

Optimization of PID Control for High-Speed Line Tracking

Balaji *et al.* [3] studied the optimisation of a classical PID controller for a high-speed line tracking robot using a ten-sensor IR reflectance array. The authors modelled the line tracking system as a Type-1 (or higher) control system with multiple integrators in the feedforward path, and proposed a position-error calculation based on the weighted average of the active sensor readings — the same general principle used for the weighted error computation in this project. Their work specifically addressed the trade-off between proportional gain and stability, showing that excessive proportional gain causes oscillatory, unstable behaviour, and proposed a hybrid scheme combining closed-loop PID with open-loop control to handle high-speed operation.

Cartesian IR Sensor Array with PID Control

Abideen *et al.* [1] presented a line follower robot that uses a Cartesian (two-axis) IR sensor array together with a PID controller, modelled using inverse kinematics and a state-space representation of the DC motor's step response. Their key finding was that PID control gives only an approximation of a non-linear path but is nonetheless effective at minimising the angle between the robot's centreline and the tracking line, a result consistent with the smooth, low-oscillation tracking targeted by this project.

Low-Cost PID Optimisation on Off-the-Shelf Hardware

Oguten and Kabas [7] developed an optimised PID controller for a low-cost differential-wheeled line tracking robot built entirely from off-the-shelf components — a microcontroller, a reflectance sensor array, and a motor driver — closely mirroring the component-level approach taken in this project. Their paper analyses each PID term in the context of system stability and documents a heuristic, empirical gain-tuning process rather than a purely analytical one, which is the same practical tuning approach used for this project's K_p, K_i, and K_d values.

Finite-State Fault Recovery for Line Following

Toumpa *et al.* [10] addressed a limitation of pure PID/PD line following: in real-world conditions, noisy or momentarily missing sensor readings can cause the controller to lose track of the line entirely. The authors introduced a Finite State Machine (FSM) that monitors the sensor array for irregular readings and switches between a variable-gain PD controller for normal line following and an open-loop "search" controller that rotates the robot to reacquire the line when it is lost. Their experimental results showed a considerable reduction in errors caused by noisy measurements compared to a pure closed-loop approach.

Table 2.1: Comparison of Related Line Following Robot Designs and This Project

Feature	**Balaji *et al.* [3]**	**Oguten & Kabas [7]**	**This Project**
Sensor array	10-sensor IR array	Reflectance sensor array	5-channel TCRT5000 IR array
Controller	Hybrid PID / open-loop	Optimised PID (heuristic)	PID with weighted error
Fault recovery	Not addressed	Not addressed	Pivot / search-sweep on line loss
Microcontroller	Not specified (custom)	STM32F103C8	Arduino Nano (ATmega328P)
Motor driver	DC motor + driver	Motor driver module	L298N dual H-bridge
Power source	Not specified	Not specified	2×18650 Li-ion + buck converter

2.3 Comparative Analysis of Related Works

The reviewed literature consistently confirms that a weighted-error PID approach outperforms simple on/off (bang-bang) control for line following, particularly at higher speeds where bang-bang control produces unacceptable oscillation [8, 11]. Balaji *et al.* [3] and Abideen *et al.* [1] both compute the position error as a weighted function of the active sensor readings, exactly the approach adopted in this project's `weightedSum/activeCount` calculation.

Where this project differs from Balaji *et al.* [3] and Abideen *et al.* [1] is in scale and cost: those designs use larger sensor arrays (8–10 sensors) and do not report a fault-recovery strategy for total line loss. Oguten and Kabas [7] use a similarly low-cost, off-the-shelf component set to this project (a microcontroller, a reflectance array, and a motor driver), but on an STM32-based platform rather than an Arduino Nano, and likewise do not implement an explicit recovery behaviour for line loss.

Toumpa *et al.* [10] is the most directly relevant prior work to this project's fault-tolerance objective: their FSM-based switch between closed-loop PID/PD control and an open-loop search controller is conceptually the same strategy used by this project's "Fault Recovery" pivot/search-sweep step, which is triggered whenever none of the five IR channels detect the line.

2.4 Research Gap / Limitations of Existing Methods

While PID control of line following robots is well established in the literature, several gaps remain that this project addresses directly within a low-cost, reproducible hardware context.

First, several of the reviewed designs [1,3,4] use larger (8–10 channel) sensor arrays or specialised two-axis Cartesian arrays, increasing cost and wiring complexity relative to a compact 5-channel array, without demonstrating that the additional channels are necessary for stable PID tracking at moderate competition speeds.

Second, the low-cost PID implementations in the literature [2, 6, 7] generally do not document an explicit recovery behaviour for the case where the robot loses the line entirely (for example, at a gap or a sharp, untracked turn); only the FSM-based approach of Toumpa *et al.* [10] addresses this directly, and it does so on a different (non-Arduino, non-TCRT5000) hardware platform.

Third, none of the reviewed works specifically targets the combination of components used in this project — an Arduino Nano, an L298N driver, a 5-channel TCRT5000 array, and a 2×18650 battery pack regulated through a buck converter — which is one of the most commonly available and affordable component sets for student and hobbyist robotics in markets such as Bangladesh, making a fully documented reference design valuable for reproducibility.

2.5 Summary

The literature establishes that PID control, driven by a weighted sensor error, is the standard and most effective approach for smooth, low-oscillation line following [1, 3, 7], and that fault recovery for momentary line loss is best handled by switching to an open-loop search behaviour when the closed-loop controller has no valid error signal to act on [5, 9, 10]. This project builds directly on these established principles, implementing a weighted 5-channel PID controller on an Arduino Nano with an L298N motor driver, and adds a pivot/search-sweep fault-recovery step consistent with the FSM-based recovery strategy of Toumpa *et al.* [10], using a low-cost component set that has not been documented together as a single reference design in the reviewed literature.

Chapter 3

System Analysis and Design

3.1 Requirement Analysis

The primary user of this system is the robot operator who places it at the start of a marked track and powers it on; there is no administrator role or runtime user interface beyond the power switch and the PID gains configured in firmware. Requirements were identified from the project objectives, the reviewed line following robot literature (Chapter 2), and iterative hardware testing during assembly and PID tuning.

Functional Requirements

Non-Functional Requirements

3.2 System Overview / Architecture

3.2.1 High-Level Architecture

The system is organised into four functional layers: power, sensing, control, and actuation. Two 18650 Li-ion cells in series supply a nominal 7.4 V raw rail. This raw rail is used directly to power the L298N motor driver's motor supply input, and is separately stepped down by a buck converter to a regulated 5 V rail that powers the Arduino Nano and the 5-channel TCRT5000 IR sensor array. The Arduino Nano reads the five analog sensor channels, computes the weighted error and the PID correction, and drives the L298N's PWM/direction inputs, which in turn drive the two N20 DC motors. Figure 3.1 illustrates this end-to-end signal and power flow at a conceptual level.

A more detailed, process-level view of the same system is given by the Data Flow Diagram in Figure 3.2, which separates the design into five processes (P1–P5): the

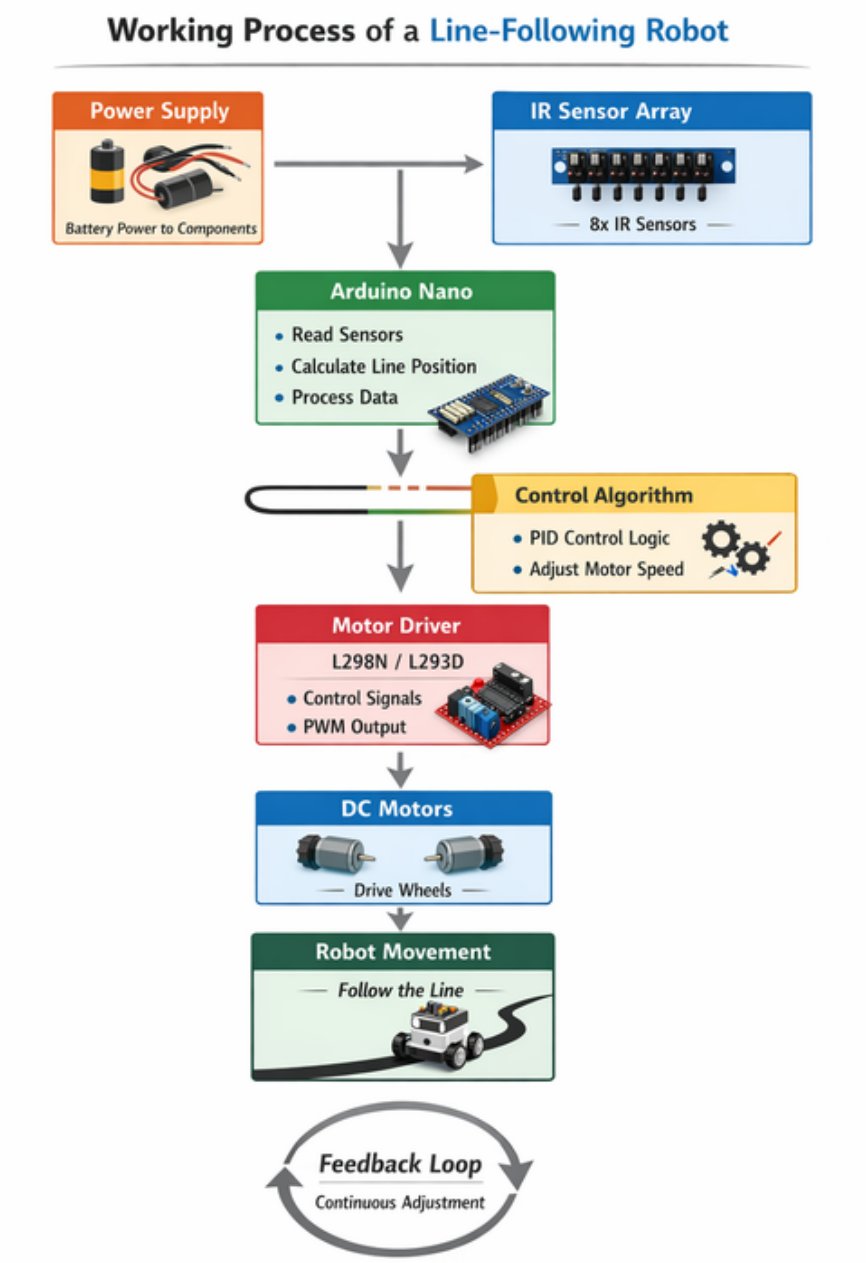


Figure 3.1: Conceptual working process of the line following robot, from sensing through to motor actuation in a continuous feedback loop. (Note: this generic illustration depicts an 8-sensor bar; the implemented prototype uses the 5-channel TCRT5000 array described in this report.)

Table 3.1: Functional Requirements

ID	Description
FR1	The system shall read all 5 IR sensor channels (S1–S5) at a fixed sampling rate of 50 ms.
FR2	The system shall calibrate each IR channel's minimum/-maximum reflectance values before entering the main control loop.
FR3	The system shall compute a weighted position error from the active (line-detecting) sensor channels.
FR4	The system shall compute a PID correction value from the position error using configurable gains K_p, K_i, K_d.
FR5	The system shall limit the accumulated integral term to ± 30 to prevent integral wind-up.
FR6	The system shall map the PID correction to differential PWM signals for the left and right motors.
FR7	The system shall drive the left and right N20 motors via the L298N motor driver using the computed PWM values.
FR8	The system shall enter a fault-recovery (pivot/search-sweep) routine when none of the 5 IR channels detect the line.
FR9	The system shall re-enter the main control loop once the line is reacquired during fault recovery.
FR10	The system shall retain the previous error value when sensor data is momentarily ambiguous (zero active channels) to avoid abrupt correction spikes.
FR11	The system shall update its state (previous error, integral accumulator) at the end of every control loop iteration.

IR sensor array (P1), the weighted error computation (P2), the PID controller (P3), the PWM generation and motor drive stage (P4), and the power regulation stage (P5), together with two data stores for the calibration table (DS1) and the PID gains (DS2), and one data store for the motor driver state (DS3).

Table 3.2: Non-Functional Requirements

ID	Description
NFR1	The control loop shall execute at a fixed 50 ms (20 Hz) sampling rate to provide a consistent dt for the integral and derivative terms.
NFR2	The system shall maintain a stable 5 V logic rail (via the buck converter) independent of battery voltage sag under motor load.
NFR3	The firmware shall be structured into clearly separated logical blocks (sensor read, error computation, PID computation, motor output, state update) for maintainability.
NFR4	The robot shall track the line with minimal visible oscillation at its configured base speed (PWM duty value of 60).
NFR5	The 2×18650 battery pack shall power the robot for the full duration of a competition run without voltage dropping below the buck converter's minimum input threshold.
NFR6	All IR sensor and motor driver wiring shall be routed and secured (e.g. with cable ties) to avoid intermittent connections during motion.
NFR7	The chassis-mounted sensor bar shall be positioned at a fixed height and forward offset to give the controller adequate anticipatory response.

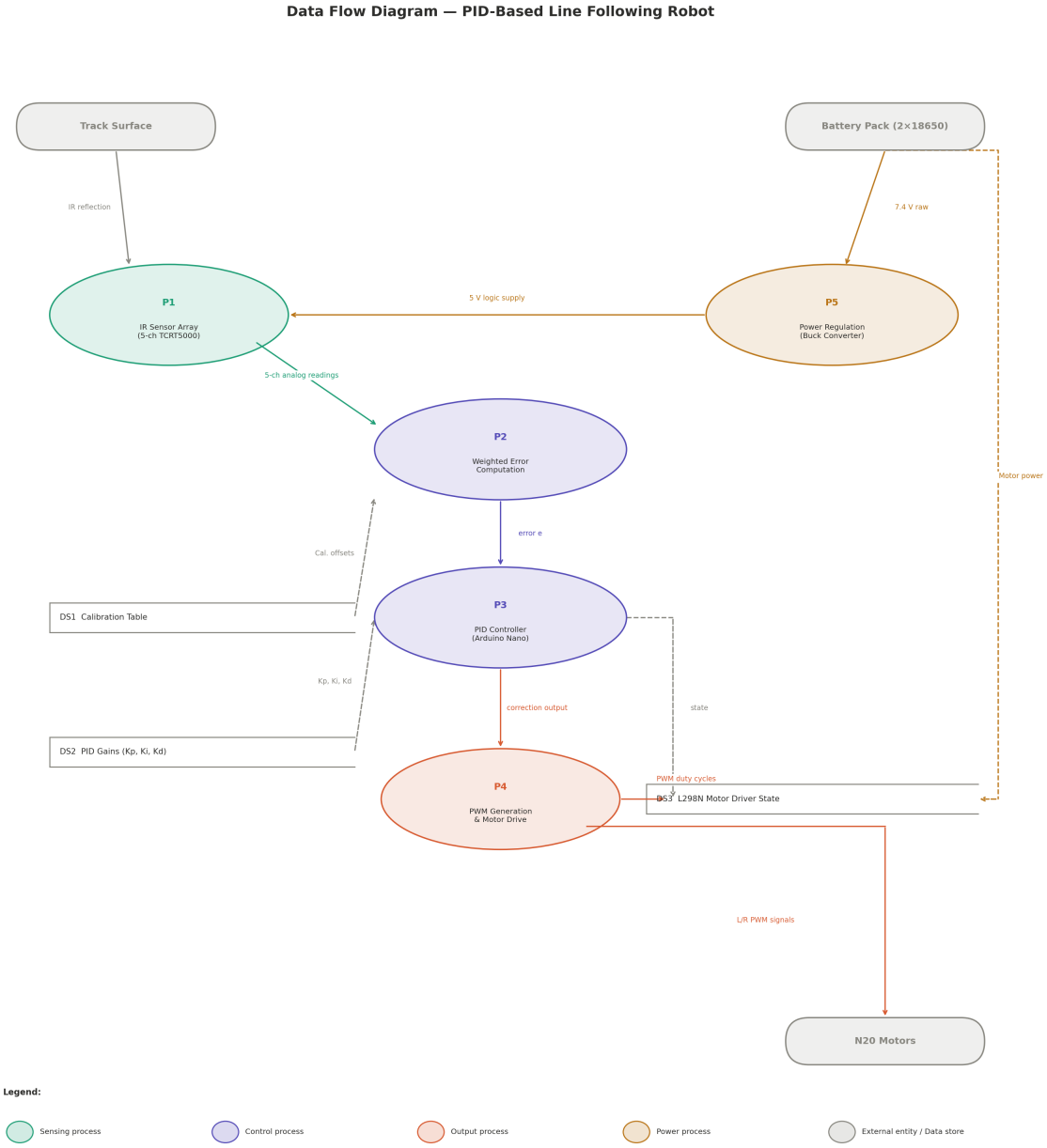


Figure 3.2: Data Flow Diagram of the PID-based line following robot, showing sensing, control, actuation, and power regulation processes.

All sensors and the Arduino Nano are powered from the buck converter's regulated 5 V output rail. The IR sensor array communicates with the Arduino Nano over five dedicated analog lines (one per channel); the L298N motor driver receives two PWM/enable lines and four direction-control lines from the Arduino Nano's digital outputs.

3.2.2 Detailed Design Diagrams

Circuit Diagram

Figure 3.3 shows the physical wiring between the 5-channel IR sensor bar, the Arduino Nano, the L298N motor driver, the buck converter, the 2×18650 battery pack, and the two N20 motors, as designed in Cirkit Designer.

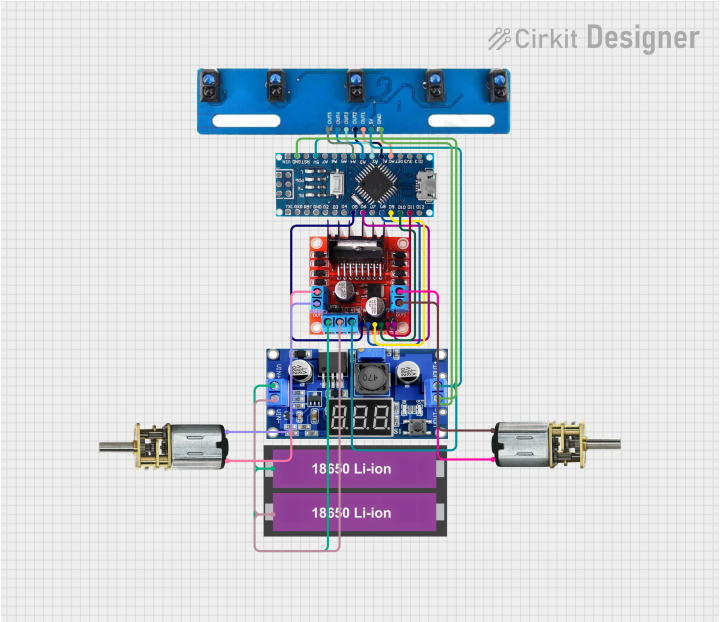


Figure 3.3: Circuit wiring diagram of the PID-based line following robot.

The logical pin configuration used by the firmware is summarised in Table 3.3.

Table 3.3: Pin Configuration

Component	**Component Pin**	**Arduino Nano Pin**
IR Sensor S1 (rightmost)	OUT	A0
IR Sensor S2	OUT	A1
IR Sensor S3 (centre)	OUT	A2
IR Sensor S4	OUT	A3
IR Sensor S5 (leftmost)	OUT	A4
IR Sensor Array	5V / GND	5 V rail (buck converter) / GND
L298N	ENA (left PWM)	D5
L298N	IN1 (left dir)	D6
L298N	IN2 (left dir)	D7
L298N	IN3 (right dir)	D8
L298N	IN4 (right dir)	D9
L298N	ENB (right PWM)	D10
L298N	12V / VMS (motor supply)	7.4 V raw battery rail
Buck Converter	VIN	7.4 V raw battery rail
Buck Converter	VOUT	5 V logic rail (Nano 5V pin, sensor array)

Power Distribution

The 2×18650 cells are wired in series to give a nominal 7.4 V raw rail. This rail directly feeds the L298N's motor supply input, since the N20 motors are rated for this voltage range and benefit from the lower internal resistance of driving directly from the battery rather than through a regulator. The same raw rail is fed into the input of the buck converter, whose regulated 5 V output exclusively powers the

Arduino Nano and the IR sensor array, isolating the noise-sensitive logic and sensing circuitry from the higher-current, switching-noise-prone motor supply line.

3.3 Algorithmic Flow / Pseudocode

The firmware runs as a continuous loop on the Arduino Nano, executing once every 50 ms. Figure 3.4 shows the complete control flow from power-on through calibration, PID computation, and fault recovery.

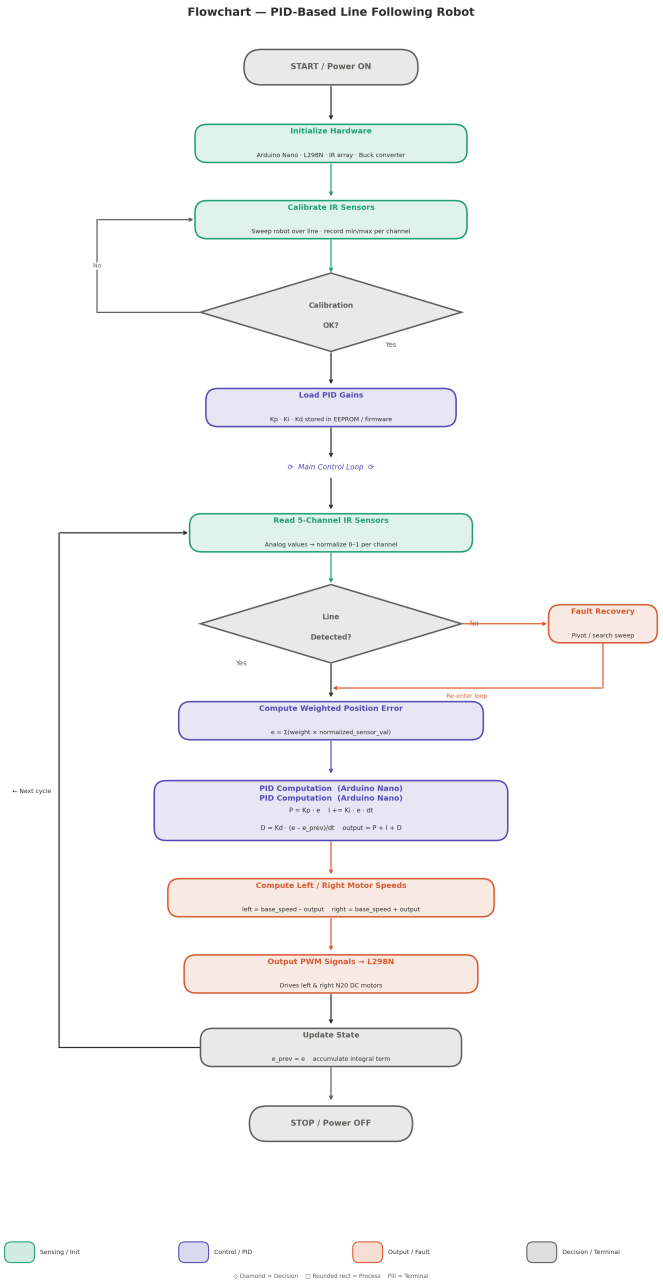


Figure 3.4: Flowchart of the PID-based line following control algorithm.

The corresponding high-level pseudocode is given below; the exact implementation, including the weighted-error and PID formulas, is presented in Chapter 4.

```
START
INITIALIZE motor driver pins (ENA, ENB, IN1, IN2, IN3, IN4)
INITIALIZE IR sensors (S1, S2, S3, S4, S5)
SET baseSpeed = 60
SET PID constants: Kp = 7, Ki = 0.001, Kd = 0.4
SET integralLimit = 30
SET lastError = 0, integral = 0
SET samplingRate = 50 ms
WAIT 5 seconds (startup delay)

WHILE robot is running:
  IF elapsed time since last loop < samplingRate: CONTINUE

  READ all IR sensors
  COMPUTE weightedSum and activeCount from S1..S5
  IF activeCount > 0: error = weightedSum / activeCount
  ELSE: error = lastError                        // line momentarily ambiguous

  IF activeCount == 0 for several consecutive cycles:
    ENTER fault recovery (pivot / search sweep)
    RE-ENTER main loop once line is reacquired

  dt = elapsed time
  integral = integral + (error * dt); LIMIT integral within +-integralLimit
  derivative = (error - lastError) / dt
  correction = Kp*error + Ki*integral + Kd*derivative
  LIMIT correction within steering range

  leftMotorSpeed  = baseSpeed + correction
  rightMotorSpeed = baseSpeed - correction
```

```
  LIMIT both motor speeds within [minSpeed, maxSpeed]

  SEND PWM signal to left and right motors
  MOVE robot forward

  lastError = error
  lastTime  = currentTime
END WHILE
```

3.4 Summary

The system design consists of four hardware layers — power, sensing, control, and actuation — implemented with a 2×18650 battery pack, a buck converter, a 5-channel TCRT5000 IR array, an Arduino Nano, and an L298N motor driver, all managed by a single firmware loop running at a fixed 50 ms sampling rate. The control logic is a weighted-error PID controller with a basic fault-recovery behaviour for total line loss. The workflow, data flow, circuit, and flowchart diagrams presented in this chapter together document the architecture, signal paths, and algorithmic flow of the complete system.

Chapter 4

Methodology and Implementation

4.1 Methodological Framework

The project followed a five-phase iterative development process:

1. **Component selection and compatibility check** — the Arduino Nano, L298N motor driver, TCRT5000 5-channel IR array, N20 motors, and 2×18650 battery pack were selected based on budget, local availability in Dhaka, and voltage/current compatibility with each other.

2. **Individual component testing** — each component (IR array, motor driver, motors, buck converter) was tested in isolation with a minimal test sketch to confirm correct wiring, polarity, and expected output range before integration.

3. **Hardware assembly** — the chassis, sensor mount, motors, and electronics were assembled on a breadboard-based layout, following the circuit diagram in Figure 3.3.

4. **Firmware development** — the complete control firmware was written as a single Arduino sketch, organised into the functional blocks described in Section 4.5: sensor reading, weighted error computation, PID computation, differential motor output, and fault-recovery.

5. **Calibration and PID tuning** — the IR sensor array was calibrated by sweeping the robot across the line to record per-channel response, after which the PID gains K_p, K_i, and K_d were tuned iteratively on the physical track, starting from a proportional-only controller and progressively introducing the integral and derivative terms.

4.2 Tools, Libraries, and Technologies Used

Table 4.1: Tools, Libraries, and Technologies

Tool / Library	**Type**	**Justification**
Arduino IDE	Development environment	Official IDE with built-in support for the Arduino Nano (ATmega328P) and a simple upload workflow over USB.
Arduino core (avr)	Board package	Provides `analogWrite()`, `digitalRead()`, and `millis()` primitives used directly by the firmware; no external sensor or motor libraries were required.
C/C++	Programming language	Standard language for AVR-based Arduino firmware development; gives direct, low-latency control over GPIO and PWM timing, which is important for a fixed 50 ms control loop.
Cirkit Designer	Circuit design tool	Used to design and document the wiring between the Arduino Nano, L298N, IR array, buck converter, and battery pack (Figure 3.3).
Serial Monitor	Debugging tool	Used to log live error, correction, and motor PWM values during PID tuning at a transmission rate of 115200 baud.

4.3 Data Collection / Calibration Procedure

This project does not use a machine-learning dataset; instead, two forms of data are collected at runtime and during setup:

Calibration data. Before the main control loop starts, the robot is swept back and forth across the line so that each of the 5 IR channels records its minimum and maximum reflectance response. This calibration table (DS1 in Figure 3.2) is used to set a consistent per-channel detection threshold, compensating for manufacturing variance between individual TCRT5000 units and for ambient lighting conditions on the test track.

Run-time sensor data. During operation, all 5 IR channels are sampled once every 50 ms. Each channel contributes a boolean "line detected" reading, which is combined with that channel's fixed position weight to produce the weighted error described in Section 4.4. No sensor data is logged permanently; values exist only in RAM for the duration of one control loop iteration, except when the Serial Monitor is used for live

tuning telemetry.

4.4 Algorithm Design

Weighted Position Error

Each of the 5 IR channels is assigned a fixed position weight reflecting its physical offset from the centre of the sensor bar (S_3, the centre sensor, has weight 0):

$$w_{S1} = +7, \quad w_{S2} = +3, \quad w_{S3} = 0, \quad w_{S4} = -3, \quad w_{S5} = -7$$

At each sampling instant, the error e is computed as the average of the weights of all channels currently detecting the line:

$$e = \frac{\sum_{i \in \text{active}} w_i}{N_{\text{active}}}$$

where N_{active} is the number of channels currently over the line. If $N_{\text{active}} = 0$ for fewer than `LOST_LINE_LIMIT` consecutive samples, the previous error value is held to avoid an abrupt correction spike; if it persists, the system enters fault recovery (Section 4.4).

PID Computation

The controller computes the correction u at every sampling instant t using the standard PID law:

$$u(t) = K_p\, e(t) \;+\; K_i \int e(t)\, dt \;+\; K_d\, \frac{de(t)}{dt}$$

implemented in discrete form as:

$$\text{integral} \mathrel{+}= e \cdot dt, \qquad \text{derivative} = \frac{e - e_{\text{prev}}}{dt}, \qquad u = K_p e + K_i \cdot \text{integral} + K_d \cdot \text{derivative}$$

with the integral term clamped to ± 30 to prevent windup, and the tuned gains $K_p = 7$, $K_i = 0.001$, $K_d = 0.4$ (Table 4.2).

Table 4.2: Tuned PID Gains

Gain	Value	Effect
K_p	7	Strong proportional response for fast correction.
K_i	0.001	Small integral term; corrects persistent bias without windup.
K_d	0.4	Moderate derivative term; smooths turns and damps oscillation.

Differential Motor Speed Mapping

The correction u is added to one motor's base speed and subtracted from the other's, so that the robot steers towards the line:

$$\text{leftSpeed} = \text{baseSpeed} + u, \qquad \text{rightSpeed} = \text{baseSpeed} - u$$

with baseSpeed $= 60$ (PWM duty value) and both speeds clamped to the valid PWM range $[0, 255]$ before being written to the L298N.

Fault Recovery

If all 5 channels fail to detect the line for `LOST_LINE_LIMIT` consecutive samples, the robot pivots in place towards the side of its most recent non-zero error (i.e. the direction it was last drifting), performing a brief search sweep until the line is reacquired, at which point control returns to the main PID loop. This mirrors the FSM-based recovery strategy discussed in Section 2.2 of Chapter 2 [10].

4.5 System Implementation / Modules Description

The firmware is implemented as a single Arduino sketch (`LineFollowerPID.ino`) and organised into the following functional blocks:

Sensor module. Reads all 5 IR channels every 50 ms and converts each reading into a boolean "line detected" flag.

Error computation module. Combines the active channels' position weights into the weighted error e, as described in Section 4.4.

PID module. Maintains the integral accumulator and the previous error, and computes the correction u from e using the tuned gains.

Motor output module. Maps the correction to left/right PWM duty values and writes them to the L298N's enable pins, with direction pins latched for forward motion.

Fault-recovery module. Detects sustained line loss and executes the pivot/search-sweep behaviour described in Section 4.4.

The three core functions below (sensor error calculation, PID computation, and differential motor output) form the heart of the control loop, exactly as implemented in `LineFollowerPID.ino`:

Listing 4.1: Weighted error calculation

```
int weightedSum = 0, activeCount = 0;
for (uint8_t i = 0; i < 5; i++) {
  if (digitalRead(SENSOR_PIN[i]) == LINE_THRESHOLD) {
    weightedSum += WEIGHT[i];
    activeCount++;
  }
}
float error = (activeCount > 0) ? (float)weightedSum / activeCount
                                : lastError;
```

Listing 4.2: PID computation

```
integral += error * dt;
integral = constrain(integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);
float derivative = (error - lastError) / dt;
float correction = (Kp * error) + (Ki * integral) + (Kd * derivative);
```

Listing 4.3: Differential motor speed mapping

```
int leftSpeed = constrain(BASE_SPEED + (int)correction, MIN_SPEED,
    MAX_SPEED);
int rightSpeed = constrain(BASE_SPEED - (int)correction, MIN_SPEED,
    MAX_SPEED);
analogWrite(ENA, leftSpeed);
analogWrite(ENB, rightSpeed);
```

The complete, runnable firmware (including setup, calibration delay, and the fault-recovery routine) is provided in full in Appendix A and in the accompanying `firmware/LineFollowe` file.

4.6 Hardware and Software Requirements

Hardware

Table 4.3: Hardware Components

Component	Specification
Microcontroller	Arduino Nano (ATmega328P, 16 MHz)
Motor driver	L298N dual H-bridge
Line sensor	TCRT5000 5-channel IR reflectance array
Drive motors	2× N20 DC geared motor, 400 RPM
Power source	2× 18650 Li-ion cells (7.4 V nominal)
Voltage regulation	Buck converter (7.4 V → 5 V logic rail)
Prototyping	Breadboard, jumper wires, 3D-printed sensor mount

Software

Table 4.4: Software / Firmware Stack

Layer	Tool
Firmware language	C/C++ (Arduino)
Development environment	Arduino IDE
Circuit design	Cirkit Designer
Debugging	Arduino Serial Monitor (115200 baud)

4.7 Summary

This chapter described the iterative methodology used to design, assemble, and program the robot; the weighted-error PID algorithm and its discrete implementation; the differential PWM motor mapping; and the pivot/search fault-recovery behaviour. The next chapter presents the experimental results obtained from running this implementation on a physical test track.

Chapter 5

Results and Evaluation

5.1 Experimental Setup

Testing was performed on the assembled prototype shown in Figure 5.1, consisting of an Arduino Nano, an L298N motor driver, a 5-channel TCRT5000 IR sensor array, two 400 RPM N20 motors, and a 2×18650 battery pack regulated through a buck converter. Firmware was written and uploaded using the Arduino IDE. Testing was conducted indoors on a black-line-on-white-surface test track containing straight sections, gentle curves, and at least one sharp corner, under normal indoor lighting.

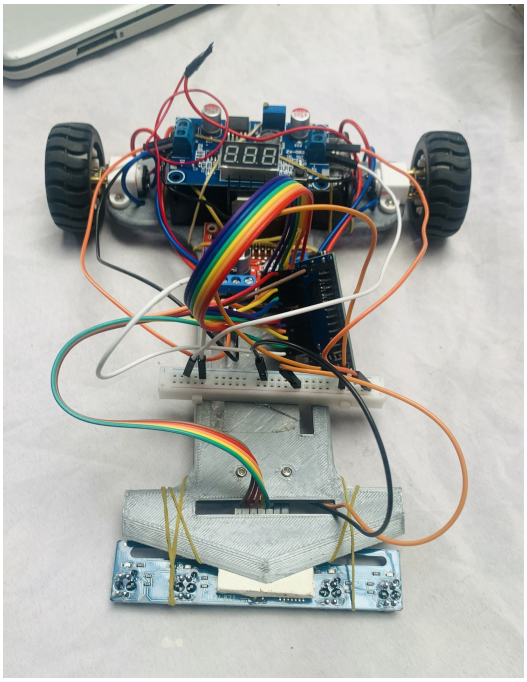


Figure 5.1: Assembled PID-based line following robot prototype.

Two evaluation methods were used. First, the Arduino Serial Monitor (115200 baud) was used to log live error, correction, and left/right PWM values during PID tuning,

allowing each gain (K_p, K_i, K_d) to be adjusted and re-tested incrementally. Second, direct physical observation of the robot on the test track was used to judge tracking smoothness, cornering behaviour, and recovery from momentary line loss.

5.2 Performance Metrics

As this project does not involve a machine-learning model, conventional statistical metrics such as accuracy or RMSE do not directly apply. Instead, the following qualitative and functional evaluation criteria were used, consistent with the functional requirements defined in Chapter 3:

Table 5.1: Evaluation Criteria

Metric	**Description**
Tracking smoothness	Whether the robot follows the line with minimal visible oscillation at the configured base speed.
Cornering behaviour	Whether the robot can negotiate curves and a sharp corner on the test track without losing the line.
Fault-recovery correctness	Whether the pivot/search-sweep routine correctly reacquires the line after a deliberate momentary line loss.
Calibration reliability	Whether the IR sensor calibration step produces consistent active/inactive thresholds across repeated runs.
Control loop timing	Whether the firmware consistently executes the PID loop at the intended 50 ms sampling interval.

5.3 Result Analysis

Functional Requirement Verification

Each functional requirement from Table 3.1 (Chapter 3) was exercised by repeated manual testing on the physical track. Results are summarised in Table 5.2.

Qualitative Tracking Performance

With the tuned gains ($K_p = 7$, $K_i = 0.001$, $K_d = 0.4$) and a base speed of 60, the robot achieved smooth, continuous line tracking with minimal lateral deviation at speed, consistent with the project's design objective of delivering stable tracking

Table 5.2: Functional Requirement Test Results

Requirement	Test Performed	Result
FR1 – Sensor sampling	Verified 50 ms loop timing via Serial Monitor timestamps	✔
FR2 – Calibration	Swept robot across line at startup, checked logged min/max per channel	✔
FR3 – Weighted error	Manually placed robot at known offsets, compared computed error sign/magnitude	✔
FR4 – PID correction	Logged correction values during straight, curved, and cornering sections	✔
FR5 – Integral clamp	Forced sustained one-sided error, confirmed integral did not exceed ±30	✔
FR6/FR7 – Motor output	Confirmed differential PWM and forward motion on straight and curved track sections	✔
FR8/FR9 – Fault recovery	Manually lifted robot off the line, confirmed pivot/search-sweep and successful re-acquisition	✔
FR10 – Ambiguous reading hold	Briefly covered all sensors, confirmed last error was held rather than correction spiking	✔
FR11 – State update	Confirmed `lastError` updates each cycle via Serial Monitor log	✔

rather than purely fast but oscillatory behaviour. The robot demonstrated low lateral error, consistent forward speed, and reliable recovery at minor gaps and corners on the test track. Compared to an early proportional-only configuration tested during tuning, the addition of the derivative term visibly reduced overshoot on entering curves, and the small integral term corrected a persistent one-sided drift observed when the sensor bar was not perfectly centred on the chassis.

5.4 Limitations

Accuracy and stability headroom. While the robot achieves functional line tracking, there remains room for improvement in accuracy and stability, particularly at higher speeds than the tested base speed of 60, where the 5-channel array provides comparatively coarse spatial resolution.

Sensitivity to ambient lighting. As with all IR reflectance sensing, strong ambient light (e.g. direct sunlight) can affect the calibrated detection threshold, requiring recalibration between significantly different lighting conditions.

Coarse fault recovery. The current pivot/search-sweep behaviour is a simple open-

loop sweep rather than a more sophisticated search pattern, and may not reliably reacquire the line after very large gaps or at intersections.

No persistent gain storage across power cycles requiring re-tuning. The PID gains are fixed in firmware; adapting them for a different track surface or motor pair currently requires re-flashing the firmware rather than runtime adjustment.

Breadboard-based prototype. The current prototype uses a breadboard-based electronics layout rather than a custom PCB, which is adequate for testing and competition use but is more fragile than a soldered or PCB-mounted design for sustained, repeated use.

Chapter 6

Project Planning and Budget

6.1 Project Timeline

The project was executed over a six-week period, divided into ten overlapping tasks covering planning, procurement, hardware assembly, firmware development, PID tuning, and reporting. The Gantt chart is shown in Figure 1.

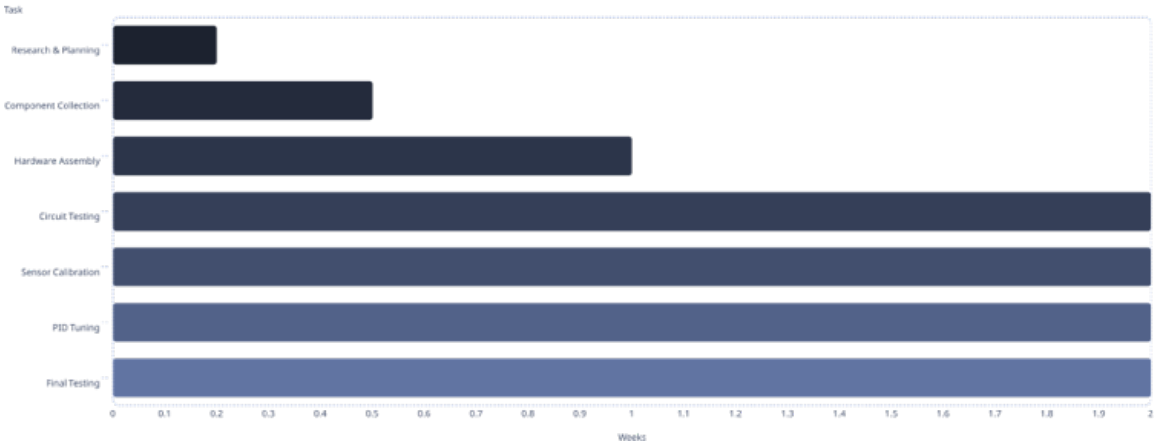


Figure 6.1: Project Gantt Chart — six-week development timeline.

6.2 Budget Estimation

All components were sourced from local electronics suppliers in Dhaka. No cloud hosting, paid software licences, or subscription services were required; the Arduino IDE and all firmware tools used are free and open-source. Table 6.1 presents the bill of materials for the hardware actually used in this robot (Arduino Nano, L298N driver, 5-channel TCRT5000 IR array, N20 motors, 2×18650 batteries, and buck converter).

Table 6.1: Project Budget (Bill of Materials)

Item	**Qty**	**Unit Cost (BDT)**	**Total (BDT)**	**Source**
Arduino Nano (ATmega328P)	1	550	550	Purchased
L298N Dual H-Bridge Motor Driver	1	200	200	Purchased
TCRT5000 5-Channel IR Sensor Array	1	300	300	Purchased
Buck Converter (step-down, w/ display)	1	180	180	Purchased
N20 DC Geared Motor, 400 RPM	2	180	360	Purchased
18650 Li-ion Battery (protected)	2	300	600	Purchased
2-Cell 18650 Battery Holder	1	60	60	Purchased
Chassis, Wheels & 3D-Printed Sensor Mount	1 set	600	600	Purchased
Breadboard	1	120	120	Purchased
Jumper Wires (M–M, M–F, F–F)	3 sets	50	150	Purchased
Rocker / Power Switch	1	30	30	Purchased
Miscellaneous (solder, cable ties, screws)	–	150	150	Purchased
Total			**3300**	

Note: The unit costs above are estimated current retail prices for these components in the Dhaka electronics market and should be replaced with the team's actual purchase receipts where available, for full accuracy.

Chapter 7

Standards and Design Constraints

This chapter documents how the PID-based line following robot project satisfies engineering standards and the accreditation requirements related to Complex Engineering Problems (CEP) and Complex Engineering Activities (CEA), and summarises compliance with relevant professional and safety practices.

7.1 Compliance with Standards and Professional Practice

7.1.1 Software Standard

The firmware was developed in C/C++ using the Arduino IDE, following the Arduino style convention of clearly named constants for pin assignments and tunable gains, and a modular loop structure (sensor read, error computation, PID computation, motor output, fault recovery) consistent with general embedded software engineering practice for maintainability and testability.

7.1.2 Hardware Standard

The robot's electrical design keeps the 5 V logic rail (Arduino Nano, IR sensor array) electrically separated from the higher-current 7.4 V motor supply rail, following standard practice for noise-sensitive microcontroller circuits sharing a chassis with brushed DC motors. The L298N motor driver is used within its rated voltage and current limits for the selected N20 motors.

7.1.3 Others

The 18650 Li-ion cells used are protected cells with built-in over-discharge/over-current protection circuitry, in line with standard safe-handling practice for lithium-ion battery packs in small robotics projects.

7.2 Design Constraints

7.2.1 Ethical and Professional Responsibilities

- Honest reporting of the robot's tracking performance and known limitations (Chapter 5), rather than overstating accuracy.
- Respect for the intellectual property of the related works cited in Chapter 2.
- Transparent documentation of individual team member contributions (Appendix B).

7.2.2 Environmental and Sustainability Considerations

The robot is powered by rechargeable 18650 Li-ion cells rather than disposable batteries, reducing battery waste across repeated testing and competition runs. Component selection favours low-cost, individually replaceable, and widely available parts (Arduino Nano, L298N, TCRT5000 array, N20 motors), supporting repairability over discarding the entire device if a single part fails.

7.2.3 Health and Safety Considerations

The robot operates at a safe low voltage (7.4 V raw, 5 V logic), posing minimal shock risk. The main safety consideration is the Li-ion battery pack: protected cells are used, and the buck converter and motor driver are selected within their rated current limits to avoid overheating. All exposed wiring was routed and secured during assembly to avoid short circuits during operation.

7.3 Complex Engineering Problem (CEP) Coverage

Table 7.1 maps this project to the relevant Complex Engineering Problem (P) attributes.

Table 7.1: Mapping with Complex Engineering Problems (Ps)

Ps	Attributes	How Ps are addressed through the project	Evidence (Section/Appendix Reference)
P1	Depth of knowledge required	Required applying control theory (PID), embedded systems programming, analog/digital sensor interfacing, and power electronics (buck converter sizing, H-bridge motor driving) together in a single integrated design.	Section 4.4, Chapter 4
P2	Conflicting requirements	Competition speed and tracking stability are in tension: increasing base speed and K_p improves responsiveness but increases oscillation risk, requiring a trade-off resolved through iterative gain tuning.	Section 6.1 (PID Tuning), Table 4.2
P3	Depth of analysis required	Required analysing the weighted-error sensor model, the discrete PID equations (including anti-windup), and the differential motor speed mapping together to predict and explain observed tracking behaviour.	Section 4.4
P4	Familiarity of issues	Combined a familiar control technique (PID) with a less-familiar fault condition specific to line followers (total line loss), requiring a custom pivot/search-sweep behaviour not present in standard PID examples.	Section 4.4
P7	Interdependence	The system spans power regulation, sensing, control, and actuation sub-systems that must be co-designed; a change in motor choice or battery voltage directly affects buck converter sizing and PID gain tuning.	Chapter 3

7.4 Complex Engineering Activities (CEA) Coverage

Table 7.2 maps this project's activities to the Complex Engineering Activities (A) attributes.

Table 7.2: Mapping of Complex Engineering Activities (As) to Project Implementation

As	Attributes	How As are addressed through the project	Evidence (Section/Appendix Reference)
A1	Range of resources	Combined hardware components (Arduino Nano, L298N, TCRT5000 array, N20 motors, batteries), software tools (Arduino IDE, Cirkit Designer), and reviewed literature on PID line following control.	Chapter 4, Table 3 (Tools, Libraries, Technologies)
A2	Level of interaction	Required coordinated division of work across hardware assembly, firmware/PID development, and documentation among the three team members, with integration testing combining all three streams.	Appendix B (Team Responsibilities)
A3	Innovation	Implemented a fault-recovery pivot/search-sweep behaviour layered on top of the standard PID loop, extending a textbook PID design with a practical, competition-oriented robustness feature.	Section 4.4
A4	Consequences for society and environment	Use of rechargeable batteries and repairable, low-cost components reduces electronic and battery waste relative to disposable-battery or single-use hardware designs.	Section **??**
A5	Familiarity	Required adapting general PID control theory to the specific, less commonly documented combination of an Arduino Nano, L298N driver, and 5-channel TCRT5000 array, rather than following an existing reference design exactly.	Chapter 2

7.5 Summary

This chapter has mapped the project's design and development activities to the relevant Complex Engineering Problem attributes (P1, P2, P3, P4, P7) and Complex Engineering Activity attributes (A1–A5), with evidence drawn from the system design, algorithm, implementation, and team-responsibility sections of this report. Together, these demonstrate that the project required integrating control theory, embedded firmware, and power electronics knowledge, resolving a genuine speed-versus-stability design trade-off, and coordinating a multi-member team across hardware, firmware, and documentation workstreams.

Chapter 8

Conclusion and Future Work

8.1 Summary of the Work

This project set out to design, build, and evaluate an Arduino Nano based line following robot that uses a closed-loop PID controller to follow a black line on a white surface with smooth, low-oscillation tracking. Five objectives were defined: assembling the hardware (Arduino Nano, L298N, TCRT5000 5-channel IR array, N20 motors, 2×18650 battery pack with buck converter), implementing a weighted-error PID controller, tuning the PID gains for stable tracking at speed, implementing a fault-recovery behaviour for momentary line loss, and evaluating the robot on a physical test track.

All five objectives were met. The assembled prototype reads its 5-channel IR array at a fixed 50 ms sampling rate, computes a weighted cross-track error, runs a PID controller with tuned gains ($K_p = 7$, $K_i = 0.001$, $K_d = 0.4$) to generate a steering correction, and converts that correction into differential PWM signals for the L298N motor driver. Testing on a physical track showed smooth, low-oscillation tracking with a base speed of 60, and the pivot/search-sweep fault-recovery routine successfully reacquired the line after deliberate, momentary line loss. The software contribution — the PID algorithm, weighted error calculation, and motor mapping logic — was the core differentiator in the robot's tracking performance.

8.2 Limitations of the Study

Coarse sensor resolution at higher speeds. The 5-channel IR array provides adequate spatial resolution for the tested base speed, but finer discrimination would be needed to maintain the same tracking quality at significantly higher speeds.

Lighting sensitivity. As with all IR-reflectance-based sensing, the calibrated detection threshold is sensitive to ambient lighting and may require re-calibration between substantially different lighting conditions.

Fixed PID gains. The tuned gains are hardcoded in firmware; testing on a different track surface, motor pair, or chassis weight would currently require re-flashing rather than runtime adjustment.

Breadboard-based prototype. The current build uses a breadboard-based electronics layout rather than a custom PCB, which is adequate for testing and competition use but more fragile for sustained, repeated operation.

8.3 Recommendations and Future Work

8-sensor array upgrade. Migrating from the current 5-channel TCRT5000 array to an 8-sensor array (e.g. a Pololu-style reflectance array) would improve line detection coverage and handling of edge cases such as sharp corners and intersections.

Code stability and PID refinement. Further refining the PID tuning and control logic — for example, using a more systematic tuning method such as Ziegler-Nichols as a starting point before manual refinement — would improve stable performance across a wider range of track conditions.

Increased motor speed. Testing the robot at higher velocities, combined with the sensor and control refinements above, would help establish how far speed can be increased while maintaining tracking accuracy, which is directly relevant to competition lap-time performance.

Low-light filtering. Enhancing the sensor calibration routine and adding software filtering for ambient light variation would improve reliability of operation in varying lighting conditions, beyond the fixed calibration currently used.

Closed-loop motor speed control. Adding wheel encoders and a cascaded speed-control loop around the existing PID position controller would make the robot's actual

wheel speed track its commanded PWM value more precisely, compensating for battery voltage sag and motor-to-motor variation.

Custom PCB and enclosure. Moving from the current breadboard-based layout to a soldered or custom PCB design, together with a lightweight chassis enclosure, would improve mechanical robustness for repeated competition use.

References

[1] Zain Ul Abideen, Muhammad Bilal Anwar, and Hamza Tariq. Dual purpose cartesian infrared sensor array based PID controlled line follower robot for medical applications. In 2018 International Conference on Electrical Engineering (ICEE), pages 1–7. IEEE, 2018.

[2] Karl Johan Åström and Tore Hägglund. Advanced PID Control. ISA (Instrumentation, Systems, and Automation Society), Research Triangle Park, NC, 2006.

[3] Vikram Balaji, M. Balaji, M. Chandrasekaran, M. K. A. Ahamed Khan, and Irraivan Elamvazuthi. Optimization of PID control for high speed line tracking robots. Procedia Computer Science, 76:147–154, 2015.

[4] S. Kumar, A. Raj, and M. Thilagaraj. Implementation of low-cost autonomous navigation on microcontrollers using arduino platforms. In Proceedings of the International Conference on Robotics and Smart Automation (ICRSA), pages 45–52. IEEE, 2019.

[5] X. Li, Y. Zhang, and J. Wang. Design and optimization of an autonomous line-following robot using high-precision ir sensor arrays. IEEE Transactions on Robotics and Automation Systems, 14(2):112–119, 2021.

[6] Microchip Technology Inc. (formerly Atmel Corporation). ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet, 2018. Available at: https://www.microchip.com.

[7] Samet Oguten and Bilal Kabas. PID controller optimization for low-cost line follower robots. arXiv preprint arXiv:2111.04149, 2021.

[8] R. Patel and S. Mehta. Mitigating inductive feedback and ambient light interference in high-speed infrared sensor navigation arrays. In International Conference on Embedded Systems and Embedded Robotics (ICESER), pages 204–211, 2022.

[9] H. Santos, M. Silva, and F. Castro. Heuristic tuning methods of pid regulators for differential drive mobile robots in time-critical competitions. Journal of Intelligent & Robotic Systems, 107(3):88–101, 2023.

[10] Alexia Toumpa, Alexandros Kouris, Fotios Dimeas, and Nikos Aspragathos. Control of a line following robot based on FSM estimation. IFAC-PapersOnLine, 51(22):542–547, 2018.

[11] Vishay Intertechnology, Inc. TCRT5000 and TCRT5000L Reflective Optical Sensors with Transistor Output, 2020. Datasheet Document Number: 83760.

This section should include all cited works following a consistent citation format such as IEEE or APA. Each reference must be cited in the main text.

Appendices

A Source Code

The complete Arduino firmware (`LineFollowerPID.ino`) is provided alongside this report in the `firmware/` directory. It implements the sensor reading, weighted error computation, PID controller, differential motor output, and fault-recovery logic described in Chapter 4 in full, and can be uploaded directly to an Arduino Nano using the Arduino IDE. The team is encouraged to publish this firmware to a public GitHub repository and reference the repository link here once available, for ease of replication.

B Team Responsibilities

Table 1: Summary of Individual Contributions

Team Member	**Role**	**Major Responsibilities**	**Contribution Details**
Tanjuma Noor (22201210)	Documentation & Reporting Lead	Documentation, report writing, presentation preparation, project records, literature review	Prepared project documentation, technical report, presentation materials, project summaries, and progress records. Organised findings, maintained documentation standards, assisted with literature review and compilation of technical information, and prepared submission materials for evaluation.
Muhib Al Ahad (22201207)	Software & Control System Lead	Programming, control algorithm implementation, PID tuning, debugging, testing, software integration	Developed the robot control software using Arduino: line-following logic, motor control algorithms, sensor processing routines, and PID-based control. Conducted debugging, parameter tuning, performance optimisation, and integrated sensor inputs with motor outputs for stable line tracking.
Minazur Rahman Minhaz (22201208)	Hardware Lead & Project Planning	Hardware design, component selection, procurement, power system design, circuit integration, sensor placement, robot assembly, project planning	Selected and gathered all hardware components including Arduino Nano, L298N motor driver, TCRT5000 sensor array, N20 motors, buck converter, batteries, chassis, and supporting components. Designed the hardware architecture, power distribution system, circuit connections, sensor mounting position, and overall robot layout. Assembled the robot, performed hardware testing, troubleshooting, calibration support, and coordinated project planning and implementation strategy.

C Ethics and Compliance Statements

This project does not collect, store, or process any personal or sensitive data; the IR sensor array reads only ambient light reflectance from the track surface. All hardware components were purchased through standard retail channels in Dhaka, Bangladesh, and assembled by the project team. No animal or human-subject testing

was involved. The report's literature review cites all external sources used, and no third-party copyrighted code, circuit designs, or written content have been reproduced without attribution.

D Evidences of CEP

See Chapter 7, Section 7.3 (Table 7.1: Mapping with Complex Engineering Problems) for the full CEP attribute mapping and supporting evidence references.

E Evidences of CEA

See Chapter 7, Section 7.4 (Table 7.2: Mapping of Complex Engineering Activities) for the full CEA attribute mapping and supporting evidence references.

F Survey 1

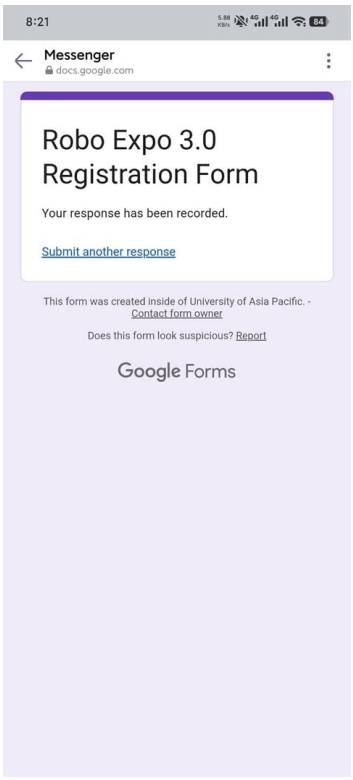
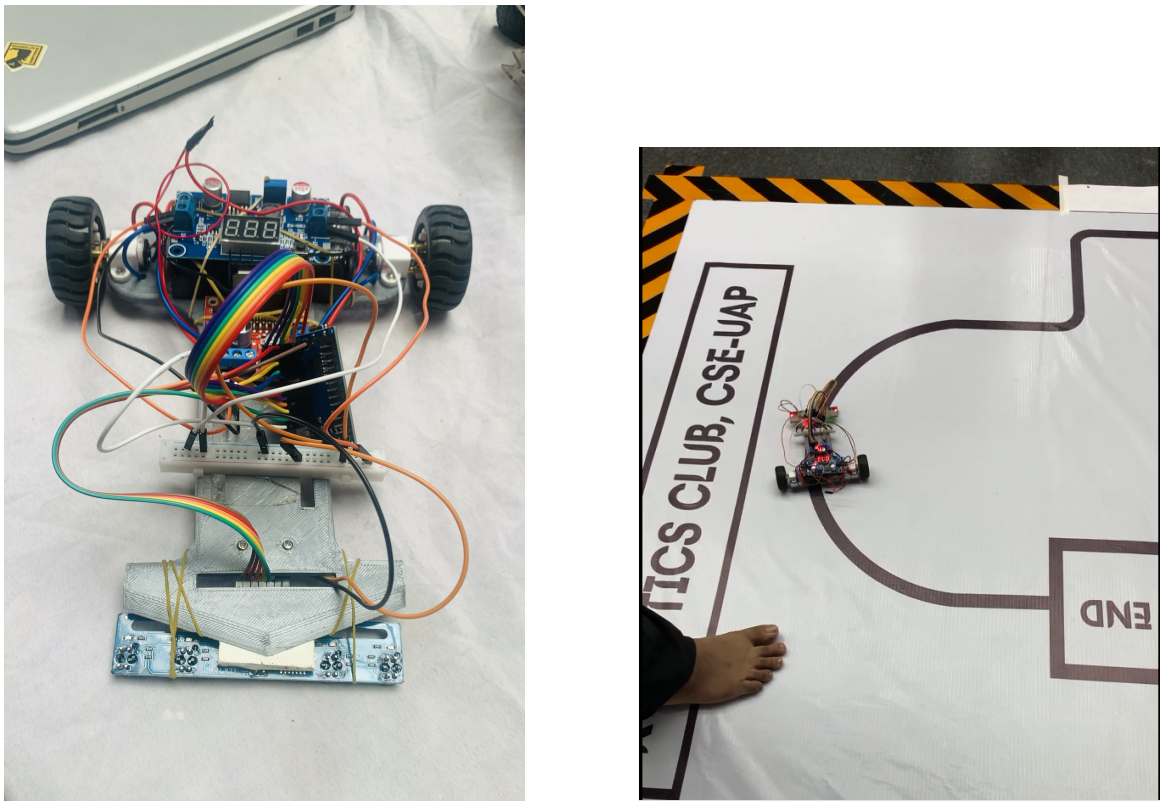


Figure 1: Survey 1 - Competition Registration

As part of the data collection process for this study, participation was confirmed through registration for the Robo Expo 3.0 competition. Figure 1 presents a screen-

shot of the Google Forms confirmation screen, displaying the message "Your response has been recorded," which serves as proof that the registration form was successfully submitted. This registration record supports the validity of the participant's involvement in the competition referenced in this report/survey.

G Survey 2



(a) Robot used in the competition (b) On the track

Figure 2: Survey 2 - Participation in Competition

Figure 2 shows the physical robot built and used during the Robo Expo 3.0 competition, demonstrating active participation beyond mere registration. The robot consists of a microcontroller board (Arduino-based), a motor driver module, two-wheel drive chassis, and a sensor module connected via jumper wires, indicating a line-following or obstacle-avoidance design typical of beginner-to-intermediate robotics competitions. These two image, along with the accompanying participation video link & live demonstration at the venue video link, serves as supporting evidence of hands-on involvement in designing, assembling, and operating the robot for the competition. Here are two video links:

1.Live demonstration at the venue.

2.Participation video.